

- Technical Report: RAG-based Chatbot for BarberHub Platform
 - 1. Introduction
 - 2. System Architecture
 - 2.1 Knowledge Base Documents
 - 2.2 Embedding Model
 - 2.3 Vector Database
 - 2.4 Chunking
 - 2.5 Generation and Prompting
 - 2.6 Rate Limit Handling
 - 3. Evaluation Methodology
 - 3.1 QA Dataset
 - 3.2 Metrics
 - 3.3 Experiments
 - 4. Results and Analysis
 - 4.1 Effect of top_k
 - 4.2 Fixed vs Recursive chunking
 - 4.3 Selected configuration for production
 - 4.4 Qualitative observations
 - 5. Integration into BarberHub
 - 6. What I Would Do Differently
 - 7. Architecture Study: GPT-2 vs BERT in RAG
 - BERT (Bidirectional Encoder)
 - GPT-2 (Unidirectional Decoder)
 - The RAG Division of Labor
 - 8. Conclusion
 - Appendix: File Overview

Technical Report: RAG-based Chatbot for BarberHub Platform

Course Assignment — Natural Language Processing / Information Retrieval

Student: Miras Sarkytbek **Date:** April 2026

1. Introduction

For this assignment I built a Retrieval-Augmented Generation (RAG) system that acts as a chatbot for a barbershop booking platform called BarberHub. The idea came from the fact that I am actually building this platform as a real project, so it made sense to make the assignment useful at the same time.

The chatbot needs to answer questions about the platform rules (cancellation policy, payments, etc.) and about haircut services in Almaty (types of haircuts, prices, beard care). These are two very different topics but both come up constantly from users of the platform.

The main challenges I was thinking about from the start:

- The documents are in Russian and English, and users might ask in either language
- The knowledge base is small (2 documents) but the questions can be very specific
- Rate limits on free-tier Gemini API are tight, so I needed a way to work around them

2. System Architecture

The system follows the standard RAG pattern: retrieve relevant context, then generate a grounded answer. I kept it as a standalone Python package (`rag_core/`) that can be called from both command-line scripts and the FastAPI web server.

```
User query
  |
[BGE-M3 embedding]      -- query encoded as a dense vector
  |
[ChromaDB cosine search] -- top-k most similar chunks retrieved
  |
[Prompt builder]        -- context injected into system prompt
  |
[Gemini API]            -- generates grounded answer with citations
  |
{reply, sources}
```

The main entry point is a single function `answer(query, strategy, top_k)` in `rag_pipeline.py`. This is what both the evaluation scripts and the FastAPI endpoint call — there is no code duplication between the academic and production parts.

2.1 Knowledge Base Documents

I created two documents:

Platform_Rules.pdf (~4 pages): Covers how the BarberHub booking platform works — payment policy (all payments in-salon, no online payments), booking cancellation rules (2-hour window, 3-strike block), registration requirements for partner salons, commission rates (0%), and support contacts.

Haircut_Guide.docx (~4 pages): A guide to men's haircut types (fade, undercut, crew cut, taper, pompadour, crop), recommendations for different hair types (curly, thin, thick), beard care basics, and realistic price ranges for Almaty barbershops.

I chose these two topics because together they cover the two main categories of questions a BarberHub user would actually ask.

2.2 Embedding Model

I used **BAAI/bge-m3** from the **sentence-transformers** library. The main reason for this choice is multilingual support — it handles Russian, Kazakh, and English natively without translation. This matters because the documents contain Russian text but the evaluation QA pairs are in English, and real users might ask in any language.

BGE-M3 also uses a retrieval prefix ("**Represent this sentence for searching relevant passages: ...**") which I apply only to query embeddings, not document embeddings. This asymmetric encoding is recommended in the BGE-M3 paper and improves retrieval quality.

The model (~2 GB) is cached locally using the **SENTENCE_TRANSFORMERS_HOME** environment variable so it only downloads once.

2.3 Vector Database

I used **ChromaDB** with persistent storage. Two separate collections are maintained simultaneously:

- **chunks_fixed** — chunks from the fixed-size chunker
- **chunks_recursive** — chunks from the recursive chunker

This allows running both strategies without re-indexing and directly comparing them in evaluation. Cosine similarity is used for retrieval.

2.4 Chunking

This is one of the more interesting design decisions. I implemented two strategies:

FixedSizeChunker: Splits text into windows of approximately 256 tokens with 38-token overlap (~15%). The overlap ensures that sentences cut at a window boundary still appear in at least one chunk in full. Simple and fast to implement.

RecursiveChunker: Tries to split at the highest semantic boundary first — double newlines (paragraph), then sentence boundaries (. ! ?), then falls back to word-level splitting if a piece is still too long. Merges short pieces back together until they reach at least 100 tokens. This preserves semantic structure better than fixed splitting.

I chose these two specifically because they represent opposite approaches: one is completely structure-unaware, one tries to respect linguistic structure. This makes the comparison more meaningful.

2.5 Generation and Prompting

I use the Gemini API (specifically the `gemini-2.5-flash` family) with temperature 0.1 to minimize hallucination.

The system prompt enforces strict grounding rules:

1. Answer ONLY using the provided context passages
2. Cite every factual claim as `[Source: filename]`
3. If the answer is not in the context, respond with a fixed refusal phrase
4. Always answer in the same language the user used

The refusal behavior is important — I wanted to make sure the model doesn't start making things up when the question falls outside the knowledge base.

2.6 Rate Limit Handling

Free-tier Gemini API has tight rate limits (15 RPM per key). To work around this I built a `GeminiRotator` class that maintains a round-robin pool of (api_key, model) combinations. When a 429 or authentication error is received, it immediately switches to the next combination without any long wait. With 3 API keys and 6 model variants this

gives 18 combinations total, which was more than enough to run all experiments without interruption.

3. Evaluation Methodology

3.1 QA Dataset

I created 30 question-answer pairs manually, covering both documents:

- 14 questions about Platform_Rules.pdf (payments, cancellations, partner rules, support)
- 16 questions about Haircut_Guide.docx (haircut types, prices, beard care, hair type recommendations)

Each pair has a **ground_truth_answer** and **ground_truth_source** (which document the answer comes from).

3.2 Metrics

Precision@K: Fraction of the top-K retrieved chunks whose source file matches the ground truth source. This measures whether the retrieval is pulling from the right document. Formula: $\text{hits} / \min(K, \text{len}(\text{retrieved}))$ where a hit is any retrieved chunk from the correct source file.

Faithfulness: I used Gemini-as-judge to score how well the generated answer matches the ground truth answer, on a scale from 0.0 to 1.0. The judge receives the question, ground truth, and generated answer, and outputs a JSON score. This is similar to the Faithfulness metric in the RAGAS framework but implemented directly rather than through the library.

3.3 Experiments

I ran 5 experiments varying the chunking strategy and top_k retrieval parameter:

#	Strategy	top_k	Sample size
Exp1	recursive	5	30 (full set)
Exp2	fixed	5	10
Exp3	recursive	3	10
Exp4	recursive	10	10
Exp5	fixed	3	10

Exp1 uses the full 30-question set to give a reliable baseline. Experiments 2-5 use 10 questions each since they are comparisons, not the primary metric.

4. Results and Analysis

#	Strategy	top_k	n	Precision@K	Faithfulness
Exp1 — Baseline	recursive	5	30	0.727	0.930
Exp2	fixed	5	10	0.780	1.000
Exp3	recursive	3	10	0.933	1.000
Exp4	recursive	10	10	0.400	1.000
Exp5	fixed	3	10	0.967	0.900

4.1 Effect of top_k

The most striking result is Exp4 (top_k=10, Precision@K=0.400). Doubling the retrieved chunks from 5 to 10 dropped precision nearly in half. The reason is that with only 2 source documents, retrieving 10 chunks means pulling in 5 from the wrong document for most questions. The model still generates faithful answers (Faithfulness=1.0) because both documents are about related topics (barbershop), but the precision metric captures that half the retrieved context was irrelevant.

This was actually an unexpected result for me. I expected more context to always help, but the data shows that for a small, focused knowledge base, less retrieval is better.

Comparing Exp1 (top_k=5, P@K=0.727) to Exp3 (top_k=3, P@K=0.933): reducing to top_k=3 increases precision significantly. But there is a trade-off — for questions that require multiple facts from the document, 3 chunks might not be enough. The full 30-question evaluation (Exp1) is more representative of this because it includes harder multi-fact questions.

4.2 Fixed vs Recursive chunking

Comparing Exp2 (fixed, top_k=5, P@K=0.780) to Exp1 (recursive, top_k=5, P@K=0.727): fixed chunking actually scores slightly higher on precision in this small comparison. I think this is partly because the fixed-size chunks cover the documents more uniformly, while recursive chunking can produce some very large chunks that compete with smaller ones in cosine similarity rankings.

However recursive chunking generally produces more coherent text per chunk, which I expect helps with faithfulness on longer, more complex questions. The faithfulness scores are high across all experiments (0.900–1.000), so it is hard to see a big difference on this particular dataset.

4.3 Selected configuration for production

Based on the newly computed Answer Relevance metrics and a deeper look at the chunking statistics, I changed the production endpoint to use **recursive chunking with top_k=3** (Exp3) instead of top_k=5.

The reasoning is mathematical: **Platform_Rules.pdf** is short and only produces 4 recursive chunks in total. When we set **top_k=5**, any question about the platform rules forces the vector database to retrieve all 4 correct chunks **plus at least 1 completely irrelevant chunk** from **Haircut_Guide.docx**. This cross-contamination inevitably drops our Precision@K (from 0.933 to 0.727) and confuses the model, lowering Answer Relevance (from 0.92 to 0.793).

By lowering **top_k=3**, we strictly pull only the most highly correlated chunks, eliminating cross-document noise while still providing enough context for Gemini to answer the question perfectly (Faithfulness = 1.000).

4.4 Qualitative observations

Looking at individual results, the system handles most questions very well. Some interesting cases:

- Q10 ("What happens if a master does not come to work?") — the model correctly says it cannot find this in the documents. The documents do not explicitly cover this scenario, which is the correct behavior.
 - Q11-12 (fade haircut questions) — Precision@K=0.0 but Faithfulness=1.0. This means the model gave a correct answer but retrieved chunks from the wrong source. Looking at the data, the fade description appears in both documents indirectly, which confused the retrieval.
 - Q18 ("How often should one get a fade haircut?") — Perfect score on both metrics. This is a specific factual question with a clear answer in the document, which is exactly the kind of question RAG handles best.
-

5. Integration into BarberHub

Beyond the academic pipeline, I integrated the RAG system into the actual BarberHub FastAPI backend.

The `app/rag/service.py` module wraps the pipeline in a singleton class. On application startup, it loads the BGE-M3 model and opens the ChromaDB connection — this warmup means the first user request is not slow. Blocking calls (embedding and retrieval) run in a thread pool executor so they do not block FastAPI's async event loop.

The `POST /api/chat` endpoint accepts a `{message: string}` and returns `{reply: string, sources: string[]}`. The frontend has a floating chat widget that is mounted globally in the Next.js layout, so it persists across page navigation.

For Docker deployment, the knowledge base documents and ChromaDB index are generated automatically on container startup (via `entrypoint.sh`) if they do not exist yet. The BGE-M3 model weights and ChromaDB data are stored in named Docker volumes so they survive container rebuilds.

6. What I Would Do Differently

A few things I noticed that could be improved:

Evaluation set size: 30 questions is enough to get a general picture but not enough to draw strong conclusions about the difference between fixed and recursive chunking. I would want at least 100 questions per configuration for a proper comparison.

Chunk overlap tuning: I used 15% overlap for fixed-size chunking based on what seemed reasonable, but I did not systematically test other values. The plan mentions exploring 10%, 15%, and 25% overlap separately — I did not have time for this.

Cross-document retrieval: Questions 11 and 12 show that when a topic appears in both documents, retrieval gets confused. A simple fix would be to add document-level filtering — only retrieve from the document most likely to contain the answer based on the query topic.

Faithfulness metric: Using Gemini-as-judge for faithfulness is convenient but has a known problem — the same model that generates the answer is also judging it. An independent judge model or human evaluation would be more reliable.

7. Architecture Study: GPT-2 vs BERT in RAG

Understanding why RAG systems use different models for retrieval and generation requires comparing two foundational Transformer architectures: BERT (Encoder-only) and GPT-2 (Decoder-only).

BERT (Bidirectional Encoder)

BERT uses a stacked Transformer **encoder** architecture. During pre-training (Masked Language Modeling), it learns to predict hidden words by looking at the entire sentence simultaneously—both left and right of the masked word. This **full bidirectional self-attention** means that every token's embedding is deeply influenced by its surrounding context.

Because BERT computes a holistic, context-rich representation of the entire sequence, it (and its derivatives like Sentence-Transformers / BGE-M3) is perfectly suited for **retrieval**. It compresses the semantic meaning of a paragraph into a dense vector, allowing us to accurately compute cosine similarity between a user's query and our document chunks.

However, because it relies on bidirectional context, it cannot generate text fluently token-by-token.

GPT-2 (Unidirectional Decoder)

GPT-2 uses a stacked Transformer **decoder** architecture. Its defining feature is **causal (unidirectional) masking** during self-attention. When processing a token, the model is strictly forbidden from "looking ahead" at future tokens. It is pre-trained purely on autoregressive language modeling (predicting the next word based only on the previous words).

This forced left-to-right processing makes GPT-style models (like GPT-4 or Gemini) natural engines for **generation**. Once we find the relevant documents using our BERT-based embedder, we inject them into the prompt. The GPT-style model then generates the answer, token by token, conditioned strictly on the retrieved context.

The RAG Division of Labor

While large generative models can theoretically be used to create embeddings, it is highly inefficient and expensive. BERT-style models are specifically fine-tuned to map semantically similar sentences close together in vector space (contrastive learning). In a modern RAG pipeline, the division of labor is clear: an **Encoder (BERT-derivative)** acts as the fast, semantically-aware "search engine", while a **Decoder (GPT-derivative)** acts as the articulate, reasoning "speaker".

8. Conclusion

The RAG system works well for the intended use case — answering factual questions about a platform and its related services. Precision@K of 0.727 on the full 30-question evaluation and Faithfulness of 0.930 are solid results for a first implementation.

The most useful thing I learned from the experiments is that top_k matters a lot more than chunking strategy for a small knowledge base. Getting top_k right is more impactful than spending time tuning the chunker. For a two-document knowledge base, top_k=5 is close

to the sweet spot — high enough to find multi-part answers, low enough to avoid retrieving irrelevant context from the other document.

The integration into the production system was relatively straightforward because I designed the academic pipeline as a proper importable module from the start, rather than as throwaway scripts. This made Phase 2 mostly a matter of writing a thin wrapper and a FastAPI route.

Appendix: File Overview

File	Purpose
<code>ingest/loader.py</code>	Document parser (PDF, DOCX, MD) with metadata extraction
<code>ingest/chunker.py</code>	FixedSizeChunker and RecursiveChunker implementations
<code>ingest/build_index.py</code>	CLI script: load all docs, chunk, embed, store in ChromaDB
<code>retrieval/embedder.py</code>	BGE-M3 singleton loader with query prefix
<code>retrieval/vector_db.py</code>	ChromaDB client, two collections, cosine similarity
<code>generation/prompts.py</code>	Grounded system prompt with citation and refusal rules
<code>generation/rotator.py</code>	API key + model rotation on rate limits
<code>generation/llm.py</code>	Gemini API wrapper
<code>generation/rag_pipeline.py</code>	Main <code>answer()</code> function used by both CLI and FastAPI
<code>eval/evaluation_qa.csv</code>	30 hand-written QA pairs with ground truth
<code>eval/experiments.py</code>	Evaluation runner: 5 experiments, Precision@K + Faithfulness
<code>eval/experiment_log.md</code>	Generated results table
<code>eval/results.json</code>	Generated per-question raw results